# Develop Natural Language Solutions in Azure

## Language, speech, and translation with Azure AI Services

Rick Beerendonk
rick@oblicum.com

glasspaper oblicum

# What You'll Learn

- Pull meaning out of text: language, sentiment, entities, key phrases, PII
- Turn speech into text and text into speech
- Decide between a purpose-built service and a large model
- Build a spoken conversation that reacts in real time
- Translate written and spoken language

# Problem: Human Language Is Not Data

A support mailbox receives 4,000 messages a week.

- Which are angry? Which mention a competitor? Which contain a customer's phone number?
- ❌ A database cannot answer any of that with a `WHERE` clause
- ❌ Reading them by hand does not scale

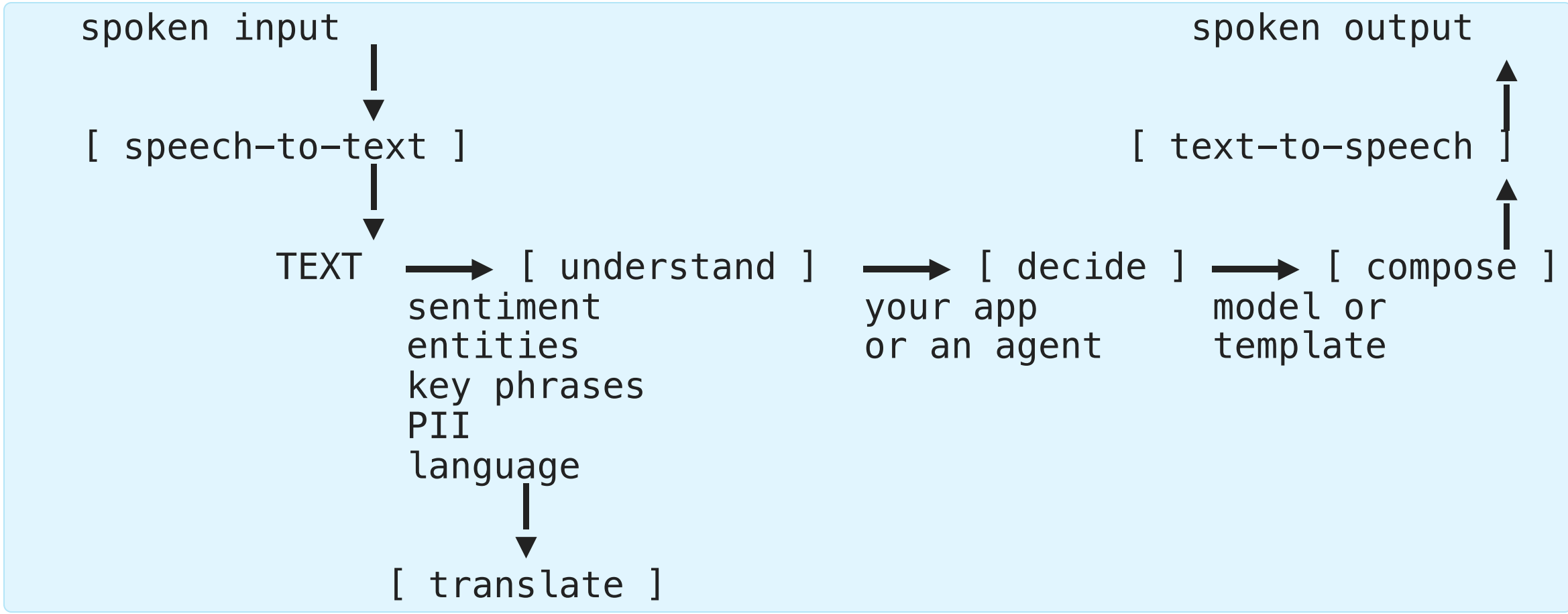
**Need:** services that convert language into structured facts you can filter, route, and store.

That is the whole point of this presentation: **language in, data out.**

glasspaper oblicum

# The Pipeline

Every solution in this presentation is a few of these boxes in a row.




**Voice Live** is this whole loop, running continuously, in under a second.

glasspaper oblicum

# Decoding the Jargon

| Term | In plain words |
|---|---|
| **NER** | Named Entity Recognition — find the names of people, places, dates |
| **Entity linking** | Also say *which* Paris — by linking to a knowledge base entry |
| **Key phrases** | The handful of noun phrases the text is actually about |
| **PII** | Personally Identifiable Information — emails, phone numbers, ID numbers |
| **Redaction** | Replacing those values with `*****` before you store or send the text |
| **STT / TTS** | Speech-to-text (listening) / text-to-speech (talking) |
| **Neural voice** | A model-generated voice, e.g. `en-US-JennyNeural` |
| **SSML** | XML that tells the voice where to pause, and in which style to speak |
| **Diarization** | Marking *who* said each sentence when several people speak |
| **VAD** | Voice Activity Detection — deciding when the person stopped talking |

# Purpose-Built Service or Large Model?

Both can read text. They fail in different ways.

| Ask yourself | Language / Speech service | Generative model |
|---|---|---|
| Do I need the *same* answer every time? | Yes — fixed categories, scores | No — wording varies |
| Do I need an open-ended answer? | No | Yes |
| Do I need a confidence score? | Yes, per result | Not really |
| Cost and latency | Low and predictable | Higher, varies with length |

**Rule of thumb:** if you can write the list of possible answers in advance, use the purpose-built service.

# Azure Language Service

glasspaper

# TextAnalyticsClient

**Package:** `pip install azure-ai-textanalytics`

```python
from azure.ai.textanalytics import TextAnalyticsClient
from azure.core.credentials import AzureKeyCredential
from azure.identity import DefaultAzureCredential

# Key-based
client = TextAnalyticsClient(endpoint="https://{resource}.services.ai.azure.com",
                             credential=AzureKeyCredential("KEY"))

# Recommended (production)
client = TextAnalyticsClient(endpoint="https://{resource}.services.ai.azure.com",
                             credential=DefaultAzureCredential())
```

**Constraints:** max 5,120 characters per document; max 1,000 items per collection

glasspaper oblicum

# Language Detection

```
result = client.detect_language(["Hello world", "Bonjour le monde"])
for doc in result:
    lang = doc.primary_language
    print(f"{lang.name} ({lang.iso6391_name}) – confidence: {lang.confidence_score}")
```

**Edge cases:**

- Multilingual text → returns dominant language with lower confidence
- Unparseable text → `language = "(unknown)"`, `score = 0`

glasspaper oblicum

# Sentiment Analysis

***[EXAM]***

```
result = client.analyze_sentiment(["I love this service!", "The wait was too long."])
for doc in result:
    print(f"Overall: {doc.sentiment}")
    print(f"  positive={doc.confidence_scores.positive:.2f}")
    for sent in doc.sentences:
        print(f"  Sentence: '{sent.text}' → {sent.sentiment}")
```

**Sentiment logic:**

- All neutral → `neutral`
- Includes positive + neutral → `positive`
- Includes negative + neutral → `negative`
- Positive + negative both present → `mixed`

glasspaper oblicum

# Entity Recognition and Linking

***[EXAM]***

## Named Entity Recognition (NER):

```
result = client.recognize_entities(["Microsoft was founded by Bill Gates in 1975."])
for doc in result:
    for entity in doc.entities:
        print(f"{entity.text} ({entity.category}) – {entity.confidence_score:.2f}")
```

**Categories:** Person, Location, DateTime, Organization, Address, Email, URL

## Entity Linking:

```
result = client.recognize_linked_entities(["I visited Paris last summer."])
for doc in result:
    for entity in doc.entities:
        print(f"{entity.name} – {entity.data_source} – {entity.url}")
```

**Entity linking** uses Wikipedia as knowledge base; disambiguates same-name entities by context (e.g., "Venus" →

# Custom Named Entity Recognition

***[EXAM]***

Train a custom entity model when the prebuilt categories do not match your domain.

**Problem:** a broad `ContactInfo` entity has **low precision** — it matches phone numbers, e-mail addresses, handles, and whatever sits near them.

**Fix the definition, not the volume:** replace it with specific `Phone`, `Email`, and `SocialMedia` entity types and relabel the training data.

| Tempting fix | Why it fails |
|---|---|
| Lower the confidence threshold | Returns **more** false positives |
| Auto-label the training data | Reproduces the same ambiguous definition |
| Add more training data | More examples of an overly broad entity |

**Rule:** low precision caused by an ambiguous label is a schema problem, not a data-volume problem.

# Retrieval Practice — Lab 1

Pause here and complete Lab 1 before continuing.

glasspaper oblicum

# Personally Identifiable Information (PII)

***[EXAM]***

**Use PII recognition to find and redact sensitive details before storage, indexing, or model input.**

```
result = client.recognize_pii_entities([
    "Contact Rick at rick@example.com or 555-0100."
])

for doc in result:
    print(doc.redacted_text)
    for entity in doc.entities:
        print(f"{entity.text} ({entity.category})")
```

**The response provides:**

- `redacted_text` with detected values replaced
- Entity text, category, offset, length, and confidence score
- Categories such as person, email, phone number, address, and government ID

**What masking actually does** — the sensitive value is **removed**, not hidden alongside the original, and each value is replaced by its **category label**:

```
Input:  Contact Rick at rick@example.com or 555-0100.
Output: Contact [PERSON] at [EMAIL] or [PHONENUMBER]
```

# Content Safety — Text Analysis

***[EXAM]***

Check user-generated text **before** it is published.

```
from azure.ai.contentsafety import ContentSafetyClient
from azure.ai.contentsafety.models import AnalyzeTextOptions

request = AnalyzeTextOptions(text=comment)
response = client.analyze_text(request)

for item in response.categories_analysis:
    print(item.category, item.severity)   # e.g. SelfHarm 4
```

| Element | Value |
|---|---|
| **Options class** | `AnalyzeTextOptions`, with the input passed as the named `text` parameter |
| **Client method** | `client.analyze_text(request)` — the synchronous `ContentSafetyClient` |

**Content Safety is not PII detection:** it classifies harmful-content severity; it does not find or mask personal data.

# Key Phrase Extraction

```
result = client.extract_key_phrases([
    "Microsoft launched its new AI product in Seattle this week."
])
for doc in result:
    print(doc.key_phrases)  # ['new AI product', 'Microsoft', 'Seattle']
```

glasspaper oblicum

# Retrieval Practice — Lab 2

Pause here and complete Lab 2, then check its solution.

glasspaper oblicum

# Language MCP Server

glasspaper oblicum

# Language MCP Endpoint and Tools

***[EXAM]***

**Endpoint:**

```
https://{foundry-resource-name}.cognitiveservices.azure.com/language/mcp?api-version=2025-11-15-preview
```

**Tools exposed as MCP:**

| Tool | Capability |
|---|---|
| Named Entity Recognition | People, places, orgs, dates, quantities |
| Sentiment Analysis | Positive/negative/neutral + aspect-level opinions |
| Summarization | Concise summaries of long text |
| Key Phrase Extraction | Main concepts and key phrases |
| PII Redaction | Detects/redacts names, addresses, phone numbers |
| Language Detection | Identifies language |

# Language MCPTool in Code

```python
from azure.ai.projects.models import MCPTool

mcp_tool = MCPTool(
    server_label="azure-language",
    server_url="https://{foundry-resource-name}.cognitiveservices.azure.com/language/mcp"
               "?api-version=2025-11-15-preview",
    require_approval="always",
)
# Optionally restrict: mcp_tool.allowed_tools = ["extract_named_entities_from_text"]
```

**Client pattern:**

```python
openai_client = project_client.get_openai_client()
response = openai_client.responses.create(
    input=[{"role": "user", "content": user_prompt}],
    extra_body={"agent_reference": {"name": "Text-Analysis-Agent", "type": "agent_reference"}},
)
print(response.output_text)
```

**Multi-task prompts:** Agent can call multiple tools in one turn (e.g., NER + sentiment)

glasspaper oblicum

# Speech with Generative AI

glasspaper oblicum

# Speech-to-Text Models (OpenAI Audio API)

```python
from openai import AzureOpenAI

client = AzureOpenAI(
    azure_endpoint=FOUNDRY_ENDPOINT, api_key=FOUNDRY_KEY,
    api_version="2025-03-01-preview"
)

audio_file = open("speech.mp3", "rb")
transcription = client.audio.transcriptions.create(
    model=MODEL_DEPLOYMENT,
    file=audio_file,
    response_format="text"
)
print(transcription)
```

**Models:** `gpt-4o-transcribe`, `gpt-4o-mini-transcribe`, `gpt-4o-transcribe-diarize`

# Text-to-Speech Models (OpenAI Audio API)

```python
with client.audio.speech.with_streaming_response.create(
    model=MODEL_DEPLOYMENT,
    voice="alloy",
    input="This speech was AI-generated!",
    instructions="Speak in an upbeat, excited tone.",
) as response:
    response.stream_to_file("speech_output.mp3")
```

**Models:** `gpt-4o-tts`, `gpt-4o-mini-tts`
**Key parameters:** `voice` (e.g., `"alloy"`), `input` (text), `instructions` (tone/style)

glasspaper oblicum

# Azure Speech SDK

glasspaper

# Choosing a Speech Capability

***[EXAM]***

| Requirement | Choose |
|---|---|
| Transcribe a **live** phone call with low latency | **Real-time speech-to-text** on the streaming audio |
| Transcribe recordings that are already complete | Batch transcription |
| Two-way spoken interaction with low latency | **Real-time speech-to-text in, text-to-speech out** |
| Produce audio from text | Text-to-speech |
| Change the language of what was said | Speech translation |

**Batch transcription waits for the recording to finish**, so it cannot serve a live transcript and it breaks turn-taking. Text-to-speech generates audio instead of transcribing it, translation changes language rather than producing a transcript, and embeddings support similarity search rather than conversation.

# SpeechConfig and Speech-to-Text

***[EXAM]***

**Package:** `azure-cognitiveservices-speech` (SDK ≥ 1.48.2)

```python
import azure.cognitiveservices.speech as speech_sdk

speech_config = speech_sdk.SpeechConfig(subscription="KEY", endpoint="ENDPOINT")

audio_config = speech_sdk.audio.AudioConfig(filename="audio.wav")
# or: use_default_microphone=True

recognizer = speech_sdk.SpeechRecognizer(speech_config=speech_config,
                                         audio_config=audio_config)
result = recognizer.recognize_once_async().get()

if result.reason == speech_sdk.ResultReason.RecognizedSpeech:
    print(result.text)
```

**`ResultReason` values:** `RecognizedSpeech`, `NoMatch`, `Canceled`
**`SpeechRecognitionResult` properties:** `Duration`, `OffsetInTicks`, `Properties`, `Reason`, `ResultId`, `Text`

glasspaper oblicum

# Text-to-Speech (SDK)

***[EXAM]***

```
audio_config = speech_sdk.audio.AudioOutputConfig(use_default_speaker=True)
speech_synthesizer = speech_sdk.SpeechSynthesizer(
    speech_config=speech_config, audio_config=audio_config
)
result = speech_synthesizer.speak_text_async("Hello world").get()

if result.reason == speech_sdk.ResultReason.SynthesizingAudioCompleted:
    print("Done")
```

**Audio format:**

```
speech_config.set_speech_synthesis_output_format(
    speech_sdk.SpeechSynthesisOutputFormat.Riff24Khz16BitMonoPcm
)
```

**Voice:**

```
speech_config.speech_synthesis_voice_name = 'en-US-Brian:DragonHDLatestNeural'
```

glasspaper oblicum

# Custom Speech

***[EXAM]***

Train a custom speech-to-text model when base recognition mishears **product names, jargon, or accents**, then deploy it to a custom endpoint.

**Workflow:** create a project → upload audio and transcripts → train → evaluate → deploy to an endpoint

```
speech_config.endpoint_id = "<Custom Speech project ID>"
```

The REST request identifies the model by the **Custom Speech project ID** — not the project URL and not the endpoint URL.

**When a custom model expires:**

| Behavior | Correct? |
|---|---|
| Requests fall back to the latest **base model** for that locale | **Yes** |
| Requests start returning 4xx errors | No |
| The expired model keeps serving until removed | No |
| The model is deleted automatically | No |

**Consequence:** recognition keeps working, but domain accuracy silently drops. Monitor expiry dates and retrain before them.

# SSML

***[EXAM]***

```
ssml = """
<speak xmlns:mstts="https://www.w3.org/2001/mstts" version="1.0" xml:lang="en-US">
  <voice name="en-US-JennyNeural">
    <mstts:express-as style="cheerful">
      Welcome to our service!
      <break time="500ms"/>
      How can I help you today?
    </mstts:express-as>
  </voice>
</speak>
"""
result = speech_synthesizer.speak_ssml_async(ssml).get()
```

**SSML capabilities:** speaking style, pauses (`<break>`), phonemes (`<phoneme>`), prosody, say-as rules, recorded audio insertion

| Requirement | Use |
| --- | --- |
| Pronounce a technical term or product name precisely | **`<phoneme>`** |

**Least effort for pronunciation is `<phoneme>`**, not a custom neural voice.

# Speech MCP Server

glasspaper oblicum

# Speech MCP Endpoint and Tools

**Endpoint:**

`https://{foundry-resource-name}.cognitiveservices.azure.com/speech/mcp`

**Two tools:**

| Capability | Details |
|---|---|
| **Speech-to-text** (Recognize) | Supports WAV, MP3, OGG, FLAC, MP4, M4A, AAC; language; phrase hints; profanity filtering (masked/removed/raw) |
| **Text-to-speech** (Synthesize) | Neural voices (e.g., `en-US-JennyNeural`); WAV/MP3 output; URL returned |

**Storage requirement:** Azure Blob Storage account required for audio I/O
**SAS URL permissions required:** Read, Add, Create, Write, List

# Speech MCPTool in Code

```
from azure.ai.projects.models import MCPTool

mcp_tool = MCPTool(
    server_label="azure-speech",
    server_url="https://{foundry-resource-name}.cognitiveservices.azure.com/speech/mcp",
    require_approval="always",
)
```

**Portal connection settings:**

- `Foundry resource name`
- `Bearer` (`Ocp-Apim-Subscription-Key`): project key
- `X-Blob-Container-Url`: SAS URL for blob container

**Prompt-based customization:** Voice selection (`using the voice "en-GB-SoniaNeural"`), language (`es-ES`), phrase hints, profanity filtering

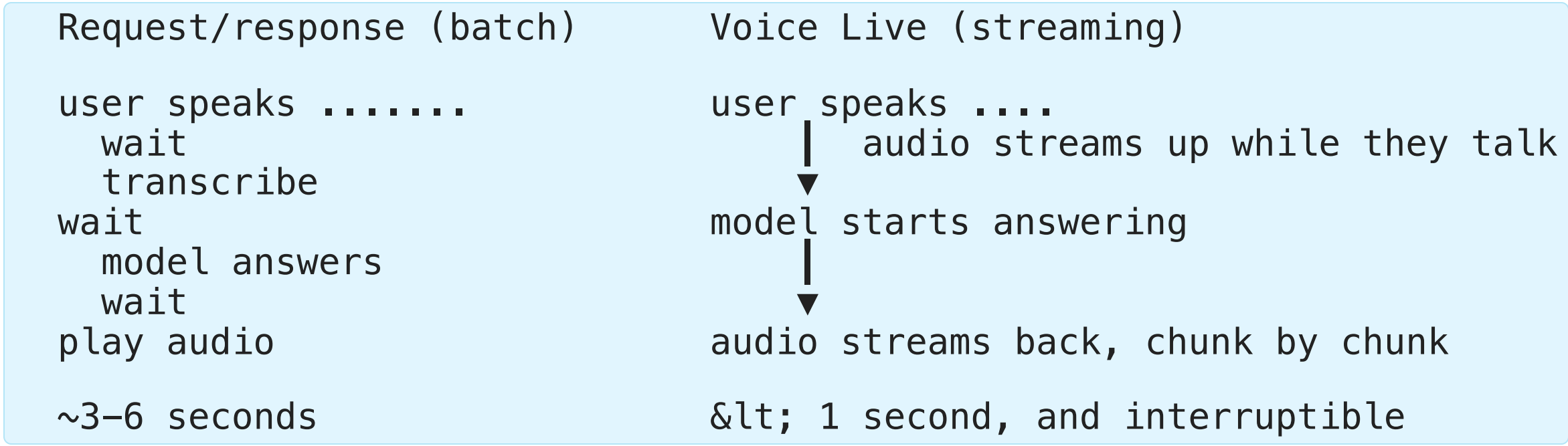
glasspaper oblicum

# Voice Live API

glasspaper oblicum

# Why Real Time Is a Different Problem

Record → transcribe → think → speak takes several seconds. In a conversation, that feels broken.




**Interruption is the hard part.** When `input_audio_buffer.speech_started` arrives, the user began talking over the answer — stop playback immediately.

**VAD** decides when a turn ended, so nobody has to press a button.

glasspaper oblicum

# Voice Live — Real-Time Speech

***[EXAM]***

**WebSocket-based, real-time bidirectional speech; low latency**

**Endpoints:**

| Connection type | Endpoint |
| --- | --- |
| Via Foundry project | `wss://{resource}.services.ai.azure.com/voice-live/realtime?api-version=2025-10-01` |

**Authentication (recommended):** Entra ID Bearer token; requires **Cognitive Services User** role
**API key auth:** `api-key` header (not browser-compatible)

**Why WebSocket:** one connection carrying bidirectional streaming audio at roughly 100 ms latency.

| Transport | Why not |
| --- | --- |
| RTMP | Built for one-way broadcast streaming |
| WebRTC | Peer-to-peer, not this client-to-service channel |

glasspaper oblicum

# Key Events

**Client → Server:**

| Event | Purpose |
|---|---|
| `session.update` | Modify session settings (voice, instructions, VAD) |
| `input_audio_buffer.append` | Add base64-encoded audio data |
| `response.create` | Trigger model inference |

**Server → Client:**

| Event | Purpose |
|---|---|
| `session.updated` | Session config confirmed |
| `response.done` | Response generation complete |
| `response.audio.delta` | Audio chunk for playback |
| `input_audio_buffer.speech_started` | User started speaking — cancel playback |
| `response.audio.transcript.done` | Transcript of AI response ready |

# Session Configuration

```
{
  "type": "session.update",
  "session": {
    "modalities": ["text", "audio"],
    "voice": { "type": "openai", "name": "alloy" },
    "instructions": "You are a helpful voice assistant.",
    "input_audio_format": "pcm16",
    "output_audio_format": "pcm16",
    "input_audio_sampling_rate": 24000,
    "turn_detection": {
      "type": "azure_semantic_vad",
      "threshold": 0.5,
      "prefix_padding_ms": 300,
      "silence_duration_ms": 500
    },
    "input_audio_noise_reduction": { "type": "azure_deep_noise_suppression" },
    "input_audio_echo_cancellation": { "type": "server_echo_cancellation" }
  }
}
```

**Python SDK:** `azure-ai-voicelive` (async-only, v1.0.0+)

glasspaper oblicum

# Retrieval Practice — Lab 3

Pause here and complete Lab 3, then check its solution.

glasspaper oblicum

# Translation

glasspaper

# Azure Translator — Text Translation

***[EXAM]***

**90+ languages; package:** `azure-ai-translation-text`

```python
from azure.ai.translation.text import TextTranslationClient
from azure.ai.translation.text.models import InputTextItem
from azure.core.credentials import AzureKeyCredential

client = TextTranslationClient(credential=AzureKeyCredential("KEY"),
                               endpoint="FOUNDRY_ENDPOINT")
# or use region: TextTranslationClient(credential=..., region="FOUNDRY_REGION")

# Detect supported languages
languages = client.get_supported_languages(scope="translation")

# Translate (auto-detects source language)
result = client.translate(
    body=[InputTextItem(text="Hola"), InputTextItem(text="こんにちは")],
    to_language=["fr", "en"]
)
for doc in result:
    print(f"Detected: {doc.detected_language.language}")
    for t in doc.translations:
```

# Transliteration and Speech Translation

**Transliterate (script conversion):**

```
result = client.transliterate(
    body=[InputTextItem(text="こんにちは")],
    language="ja", from_script="Jpan", to_script="Latn"
)
# → "Kon'nichiwa"
```

**Speech Translation (SDK):**

```
translation_cfg = speech_sdk.translation.SpeechTranslationConfig(
    subscription="KEY", endpoint="ENDPOINT"
)
translation_cfg.speech_recognition_language = "en-US"
translation_cfg.add_target_language("fr")
translation_cfg.add_target_language("ja")

translator = speech_sdk.translation.TranslationRecognizer(
    translation_config=translation_cfg,
    audio_config=speech_sdk.AudioConfig(use_default_microphone=True)
)
```

# Retrieval Practice — Lab 4

Pause here and complete Lab 4 before continuing.

glasspaper oblicum

# Azure AI Immersive Reader

***[EXAM]***

**An assistive reading experience:** read-aloud, line focus, syllable breaking, picture dictionary, and single-word translation.

**Choose it when** the requirement is **reading accessibility** — for example dyslexia support — with minimal setup.

| Not this | Because it |
|---|---|
| Document Intelligence | Extracts content from documents |
| Azure Language | Analyzes text |
| Translator | Converts languages |

None of them provide the assistive reading experience itself.

# Private Networking for a Language Resource

***[EXAM]***

Analyzing confidential **on-premises** documents:

1. Deploy the **Language resource**
2. Add a **private endpoint** — the service gets a private IP inside your virtual network
3. Connect from on-premises over **VPN or ExpressRoute**

| Alternative | Why it fails |
|---|---|
| Public endpoint | The service stays reachable over the internet |
| VPN or ExpressRoute alone | Without a private endpoint the service is not private |
| Managed identity | Controls authentication, not network routing |

**Identity answers “who”; networking answers “from where.”**

# Key Takeaways

1. **TextAnalyticsClient** — language detection, sentiment, NER, key phrases, entity linking; 5,120 char / 1,000 item limits
2. **Language MCP server** — 9 tools; agent decides which to use per prompt; `MCPTool` to attach
3. **OpenAI audio models** — `gpt-4o-transcribe/tts` for GPT-integrated speech pipelines; API version `2025-03-01-preview`
4. **Azure Speech SDK** — `SpeechConfig`, `SpeechRecognizer`, `SpeechSynthesizer`; SSML for style control
5. **Speech MCP server** — Blob Storage required for audio I/O; two tools: recognize + synthesize
6. **Voice Live API** — WebSocket real-time speech; `session.update` with VAD, noise reduction, echo cancellation
7. **Translation** — `TextTranslationClient`; `translate()` + `transliterate()`; `SpeechTranslationConfig` for spoken input

# Retrieval Practice — Lab 5

Pause here and complete Lab 5, then check its solution.

glasspaper oblicum

# Retrieval Practice — Topic 3

Pause after each matching section and answer the checkpoint questions before continuing:

- **Language analysis:** Questions 1–3
- **Safety and privacy:** Questions 4–6
- **Speech:** Questions 7–9
- **Translation and pronunciation:** Questions 10–12
- **Responsible AI:** Questions 13–15

Open Topic 3, Lab 1 · Open the Lab 1 solution

glasspaper oblicum

# Official Microsoft Slides

Open the official Topic 3 slides in original order

glasspaper oblicum